EFFICIENT COMPUTATION OF TRIPLE SCHUBERT STRUCTURE CONSTANTS

MATTHEW J. SAMUEL

Abstract. We present `schubmult`, an algorithm that expands a product of two double Schubert polynomials in different sets of variables into the double Schubert basis, and we bound its computational cost by $2^{f(n)+O(\log n)}$ coefficient-ring operations, where $f(n) = 2n\log_2 n + (3-2\log_2 e)n + \Theta(\log^2 n)$. For ordinary Schubert polynomials, the $\Theta(\log^2 n)$ term may be deleted. In the Grassmannian case (Schur and factorial Schur polynomials), the exponent collapses to $4(2n-k)H_2(k/(2n-k)) + \Theta(\log^2 n)$, with H_2 the binary entropy function. At the balanced dimension $k = n/2$, this is $2n\log_2(27/4) \approx 5.5098n$, yielding a cost of $(27/4)^{2n+o(n)}$ against $n^{\Theta(n)}$ in general; the maximum over k is $8n\log_2\phi \approx 5.5539n$ for ϕ the golden ratio, attained at $k \approx 0.553n$, so the balanced case is within one percent of the worst. The analysis rests on two structural facts: every permutation arising in the computation, output or intermediate, lies in a single interval of the left weak order, bounding their number by $(2n-1)!!$; and $\mathfrak{S}_v(x;z)$ admits a signed expansion into products of factorial elementary symmetric polynomials with variable counts λ_i for a set of admissible partitions λ, the counts being a free parameter. Because the product $\mathfrak{S}_u(x;y)\,\mathfrak{S}_v(x;z)$ has at least $|[e,v]_L|$ terms for every u, yielding $n!$ terms at $v = w_0(n)$, the cost for each fixed u is at most the cube of the worst-case output size, making `schubmult` worst-case output-polynomial. The algorithm is implemented in the Python package `schubmult`, available on the Python Package Index.

Keywords: Schubert polynomial, structure constant, Littlewood-Richardson rule, coproduct, double Schubert polynomial

Mathematics classification codes: 05E05, 05A05, 05A15, 14N10

Email: matthematics@gmail.com

1. Introduction

Multiplying two Schubert polynomials and expanding the result back in the Schubert basis is the central computational problem of Schubert calculus. The resulting structure constants c_{uv}^w are the intersection numbers of Schubert varieties in a flag manifold; in the Grassmannian case they specialize to the Littlewood–Richardson coefficients, and in the double (equivariant) case they become polynomials in two families of coefficient variables. This paper is about computing the entire expansion, and about proving what that computation costs.

1.1. The problem.

Almost all of what is known concerns a single coefficient. Computing one Littlewood–Richardson coefficient is $\#P$-complete [10]; deciding whether it is nonzero is, by contrast, in P [7, 4]. For Schubert polynomials the coefficients c_{uv}^w generalize the Littlewood–Richardson coefficients, so computing one is $\#P$-hard, while membership in $\#P$ is open, no combinatorial rule being known; the vanishing problem was recently placed in Σ_2^p [11]. These results are discussed in Remark 5.17.

None of this tells us what it costs to produce the whole product, which is what an implementation actually does. A complete expansion is not a single number but a whole table of them, and the natural accounting is not per output bit but per combinatorial witness. Such an accounting is only half of a complexity result, and earlier work has stopped at the missing half. Remmel and Whitney [12] close their paper on multiplying Schur functions with a bound linear in the number of tableaux their algorithm produces, and then stop: the first step toward a worst-case or average-case statement, they observe, is to estimate that number, and they know of no work on the question. An output-sensitive bound becomes a complexity bound only when the size of the output is itself under control. We supply both halves here. The cost bound is Theorem 5.7; the missing half is Proposition **??**, which bounds the number of terms below. It rests on Theorem 5.8, a vanishing criterion for the double structure constants which reduces every positive degree to the classical degree-0 case, and it needs from that case only the fact that a product of ordinary Schubert polynomials is nonzero—no estimate of the intractable degree-0 count enters (Remark **??**). The resulting bound is explicit and identifies which structural feature of the double case is expensive.

1.2. **Main results.** Fix $u, v \in S_n$. The algorithm **schubmult**(u, v) of Method 5.1 computes the full expansion of $\mathfrak{S}_u(x; y)\ \mathfrak{S}_v(x; z)$ in the Schubert basis. Our results about it are the following.

A support bound of $(2n-1)!!$. Let $w_0 = w_0(n)$ be the longest element of S_n and let $w_1 = w_1(n)$ be the dominant permutation with $\mathfrak{c}(w_1) = \mathfrak{c}(w_0) + \mathfrak{c}(w_0) = (2n-2, 2n-4, \ldots, 2, 0)$. Every permutation appearing in the product, or at any intermediate stage of its computation, satisfies $w \leq_L w_1$ in the left weak order, and the number of such permutations is

$$(2n-1)!! \ = \ 1 \cdot 3 \cdot 5 \cdots (2n-1).$$

This is Lemma 5.3; the proof is a Cauchy-formula factorization of $\mathfrak{S}_{w_1}$ into two copies of $\mathfrak{S}_{w_0}$, followed by linear independence of the Schubert basis. The doubled staircase w_1, and not S_n or S_{2n-1}, is what confines the computation.

An explicit complexity bound. Theorem 5.7 bounds the number of coefficient-ring operations by

$$O\big(\, n^2\, \Phi_v \ + \ L \cdot P_u \cdot B \cdot n^3 \ + \ L \cdot P_u \cdot V \cdot B \cdot \Delta \cdot E_n \,\big)\,,$$

whose three summands are the one-time v-side precomputation, the live Pieri enumeration, and the matched-pair ring work. Written as $2^{\,f(n)+O(\log n)}$, this is

$$f(n) = 2n \log_2 n + (3 - 2\log_2 e)\, n + \Theta(\log^2 n),$$

so the worst case is bounded by $n^{\Theta(n)}$; when v is dominant this improves to $f(n) = n \log_2 n + (2 - \log_2 e)\, n + \Theta(\log^2 n)$. The leading term is exactly the support bound above: the $(2n-1)!!$ permutations on the u-side and the at most $n!$ labels on the v-side multiply to $(2n)!/2^n$, and Stirling's approximation converts this into the stated closed form.

Output-polynomiality, measured at the top of a Tamari class. Theorem **??** bounds the cost by $O(n^{2L+4} N^{\uparrow}(u, v) E_n)$, where L is the number of columns processed and

$N^{\uparrow}(u, v) = N(u, \pi^{\uparrow}(v))$.

For bounded L this is polynomial in n and linear in the size it measures. That size is not quite the one an implementation reports: $N^{\uparrow}(u, v)$ counts the terms of the product at the top of the Tamari class of v rather than at v itself. The two agree exactly when v is dominant, and Corollary **??** confines their discrepancy by the spread of $N(u, -)$ over the class, every other quantity in that estimate being a class invariant. Whether the passage from $N^{\uparrow}(u, v)$ to $N(u, v)$ costs only a polynomial factor in general we do not know, and we regard it as the main question left open here. Its interest is not only bookkeeping: the classes are the fibres of a lattice congruence, so the question asks how far a Schubert product can vary along a congruence class, which seems worth understanding for its own sake.

A representation with free variable counts, and stable integer coefficients. Theorem 4.1 expands a double Schubert polynomial as a signed sum, over chains, of products of factorial elementary symmetric polynomials, and does so for any partition λ of variable counts subject only to the reachability condition $v \leq_L \mu_\lambda$, where μ_λ is the dominant permutation with $\mathfrak{c}(\mu_\lambda^{-1}) = \lambda$:

$$\mathfrak{S}_v(x; z) = \sum_{(v_0, \ldots, v_k)} \prod_{i=1}^{k} (-1)^{\sigma_i(v_{i-1}, v_i)}\, E_{\lambda_i - \ell(v_{i-1}, v_i),\ \lambda_i}\big(x; z_{D_i(v_{i-1}, v_i)}\big),$$

the chains running from the identity to $v\mu_\lambda^{-1}$. The counts are therefore a parameter of the representation rather than data extracted from v. A product of this kind is a standard elementary monomial when the counts are strictly decreasing, which is one admissible choice; the minimal choice $\lambda = \theta(v^{-1})$, which **elemrepr** uses because it minimizes the number of columns, is another. Corollary 4.2 shows that all of them carry the same integer data: setting $z = 0$ leaves the indexing set and the signs untouched, so the ordinary and double expansions are indexed by the same chains with the same integer coefficients, and one passes from the first to the second by the single substitution

$$e_p(x_1, \ldots, x_m) \ \longmapsto \ E_{p,m}\big(x; z_{D_i}\big),$$

replacing each elementary symmetric factor by the factorial elementary symmetric polynomial on the alphabet z_{D_i} that the chain already designates. Nothing combinatorial is added by the equivariant parameters; they only decorate factors that were going to be built anyway.

Where the cost of the double case sits. Corollary **??** gives the bound for ordinary Schubert polynomials, and it is that of Theorem 5.7 with the term $\Theta(\log^2 n)$ deleted. By the stability just described the two

algorithms traverse identical witnesses with identical signs, so their entire difference is the weight attached to a witness, 1 in the ordinary case against one factorial elementary symmetric polynomial in the double case. Proposition 5.6 prices that weight exactly: building $E_{p,m}$ by the divide-and-conquer recursion costs $E_n = n^{\Theta(\log n)}$ ring operations, a quasi-polynomial factor, superpolynomial in n but subexponential. The bottleneck of the double case is thus coefficient-ring arithmetic rather than combinatorial explosion, which is why the implementation keeps every coefficient in factored form and never expands it into monomials.

The quantum case. Remark 4.4 describes the quantum setting, which fits the same framework with one localized exception adopted for speed. Worked in the SEM form throughout, with strictly decreasing counts, the quantum expansion is structurally identical to the classical one: it is a signed sum of products of factorial quantum elementary symmetric polynomials, indexed and signed as before, and the stability above applies verbatim. To go faster we consolidate certain pairs of factors into the double Schubert polynomial of the single permutation they jointly correspond to, replacing two Pieri steps by one; quantumly these consolidated factors can be kept as quantum double Schubert polynomials that behave correctly. Only they leave the elementary symmetric world, and only away from $q = 0$; every other factor is a factorial quantum elementary symmetric polynomial. The Pieri formulas the consolidated factors require are new, with proofs long enough to warrant separate papers, and we forward-cite them to [14, 13], in preparation. The architecture of **schubmult** is indifferent to this substitution.

The Grassmannian collapse, in n and k. When both factors are k-Grassmannian the product is Schur, respectively factorial Schur, multiplication. Lemma 6.2 confines every intermediate object to the $k \times 2(n-k)$ box, so both support counts and both per-step branchings drop to $G_k = \binom{2n-k}{k}$, and Theorem 6.3 and Corollary 6.5 reduce the exponent to

$$4(2n-k)\, H_2\!\left(\frac{k}{2n-k}\right), \qquad H_2(x) = -x\log_2 x - (1-x)\log_2(1-x).$$

The constant 4 is retained deliberately, as a big-O reading would discard it. In the balanced case $k = n/2$, $G_{n/2} = \binom{3n/2}{n/2} = (27/4)^{n/2+o(n)}$ and the exponent is $2n\log_2(27/4) = 5.5098\ldots n$, so the cost is $(729/16)^{n+o(n)}$. This is close to the worst case over k: the maximum over all k is $8n\log_2\varphi \approx 5.554\,n$, with $\varphi = (1+\sqrt{5})/2$ the golden ratio, attained near $k \approx 0.553\,n$, less than one percent higher. Only the degenerate dimensions do better, the exponent shrinking to $\Theta(\log^2 n)$ as $k \to 0$ or $k \to n$, mirroring the shrinking of $\mathrm{Gr}(k,n)$ at the extremes. The running time is thus $2^{O(n)}$ for every k, where the general bound is $n^{\Theta(n)}$: a saving of a factor $n^{\Theta(n)}$. Since $G_k = \Theta(n^k)$ for fixed k, this is also where the familiar statement that the count is polynomial in n comes from; Remark 5.17 explains why a bound of this shape, rather than polynomiality in the binary input size, is the most that can be asked here, and how far it sits from the information-theoretic floor.

1.3. **The algorithm.** The computation rests on the expansion of Theorem 2.3, which writes a double Schubert polynomial as a signed sum of products of factorial elementary symmetric polynomials indexed by chains, together with the Pieri formula of Theorem 2.4. It is assembled from three routines, each analyzed separately.

pieri (Section 3) enumerates the u-side covers of a single Pieri step. The enumeration is multiplicity-free (Lemma 3.3), and costs $O(n^3)$ per permutation produced (Proposition 3.4).

elemrepr (Section 4) builds the v-side, and is where Theorem 4.1 and its stability corollary are proved. A dominant double Schubert polynomial factors, one factorial elementary symmetric polynomial per column, and a general v is reached from a dominant one by divided differences applied column by column. Carried out naively this signed descending expansion produces large numbers of terms that eventually cancel. The dominant permutation itself supplies the filter that discards them: only endpoints beneath it in weak order are retained (Lemma **??**), so no cancelling label ever seeds the next column and the number of live paths does not blow up.

schubmult (Section 5) yokes the two sides. It proceeds in layers, one per column of the shape, maintaining coefficients indexed by pairs (intermediate u-side permutation, v-side label). Two features drive the analysis. The v-side is precomputed once, so its transitions are table lookups; and within a layer the Pieri enumeration is run once per distinct u-side permutation rather than once per key, which decouples it from the number of v-side labels. This is what splits the cost into the three independent summands of Theorem 5.7.

1.4. **Implementation.** The three routines are not idealized descriptions of a computation carried out on paper. They are the routines of a program: **schubmult** is implemented in the Python package of the same name [16], distributed through the Python Package Index and installable in the usual way,

```
pip install schubmult
```

with sources and a command line interface included. The names **pieri**, **elemrepr** and **schubmult** are the names of the corresponding components there, so the complexity statements proved below are statements about code that exists and runs rather than about a design.

We record this for two reasons. The bounds of Theorem 5.7 count coefficient-ring operations, and a reader who wishes to see how that count behaves against wall-clock time, or on inputs beyond the reach of hand computation, can simply run the program. More importantly, every structural claim made here has a computational counterpart that the package makes checkable: the confinement of intermediates to a single weak order interval (Lemma 5.3), the coefficient stability of Corollary 4.2, the freedom in the counts λ afforded by Theorem 4.1, and the collapse of the Grassmannian case (Section 6) can each be verified directly on any given input. We have used it throughout in exactly that way.

1.5. **Organization.** Section 2 fixes notation and records the two formulas the algorithm is built on. Sections 3, 4 and 5 present **pieri**, **elemrepr** and **schubmult** with their correctness and cost statements, and the complexity analysis is assembled in Theorem 5.7 and Corollary **??**, with prior work discussed in Remark 5.17. Section 6 treats the Grassmannian case. The implementation is described in Section 1.4.

2. Foundational formulas

2.1. Notation and definitions.

Definition 2.1 ($\mathfrak{c}$, dominant permutations, θ, $\pi^\uparrow$)**.** We say that a permutation μ is *dominant* if $\mathfrak{c}_i(\mu) \geq \mathfrak{c}_{i+1}(\mu)$ for all i. For a permutation $v \in S_\infty$, we define a dominant permutation $\pi^\uparrow(v)$ recursively as follows. If v is already dominant, define $\pi^\uparrow(v) = v$. Otherwise, let i be the maximal index such that $\mathfrak{c}_i(v) < \mathfrak{c}_{i+1}(v)$, and define

$$\pi^\uparrow(v) = \pi^\uparrow(vs_i)$$

Define $\theta_i(v)$ to be $\mathfrak{c}_i(\pi^\uparrow(v))$.

Definition 2.2 ($\overset{i}{\Rightarrow}$, $\sigma_i(u,v)$, $D_i(u,v)$)**.** Let $u, v \in S_\infty$ and $i \geq 1$ be an integer. We define $u \overset{i}{\Rightarrow} v$ if there exist distinct integers $b_1, b_2, \ldots, b_k$ such that $b_j \neq i$ for all j, $\ell(ut_{i,b_1} \cdots t_{i,b_j}) = \ell(u) + j$ for all j, and

$$v = ut_{i,b_1} \cdots t_{i,b_k}$$

We define

$$\sigma_i(u,v) = \#\{j \mid b_j < i\}$$

and we define

$$D_i(u,v) = \{u(i)\} \cup \{u(j) \mid u(j) \neq v(j)\}$$

2.2. Dominant fiddling.

Suppose $u \leq_L \mu_A$ and $v \leq_L \mu_B$. Then

$$\begin{aligned}
&\mathfrak{S}_{\mu_A}(x; y_A)\, \mathfrak{S}_{\mu_B}(x; y_B) \\
&= \sum_{u,v} \mathfrak{S}_u(x; a)\, \mathfrak{S}_v(x; b)\, \mathfrak{S}_{\mu_A u^{-1}}(a; y_A)\, \mathfrak{S}_{\mu_B v^{-1}}(b; y_B) \\
&= \sum_{w} c_{u,v}^w(a; b)\, \mathfrak{S}_w(x; a)\, \mathfrak{S}_{\mu_A u^{-1}}(a; y_A)\, \mathfrak{S}_{\mu_B v^{-1}}(b; y_B)
\end{aligned}$$

2.3. The formula for double Schubert polynomials.

Theorem 2.3 (Formula for double Schubert polynomials)**.** *Suppose $v \in S_\infty$. Then*

$$\mathfrak{S}_v(x; z) = \sum_{(v_0, \ldots, v_k)} \prod_{i=1}^{k} (-1)^{\sigma_i(v_{i-1}, v_i)} E_{\theta_i(v) - \ell(v_{i-1}, v_i), \theta_i(v)}(x; z_{D_i(v_{i-1}, v_i)})$$

Where $v_{i-1} \overset{i}{\Rightarrow} v_i$ for all i, $v_0 = 1$, and $v_k = v\pi^\uparrow(v^{-1})$.

The counts $\theta_i(v)$ appearing here are the minimal admissible ones. Theorem 4.1 below shows that the formula persists with $\theta(v)$ replaced by any partition λ that reaches v, the endpoint changing accordingly; the statement above is the case $\lambda = \theta(v^{-1})$.

2.4. **The Pieri formula.**

Theorem 2.4 (The Pieri formula, [15, Theorem 7.1])**.** *Suppose $u \in S_\infty$, $k \geq 1$, and $0 \leq p \leq k$. Let x, y, z be sets of variables. Then*

$$\mathfrak{S}_u(x;y)\, E_{p,k}(x;z) = \sum_{u \xrightarrow{k} w} E_{p-\ell(u,w),k-\ell(u,w)}(y; z_{P_k(u,w)})\, \mathfrak{S}_w(x;y)$$

where $P_k(u,w) = \{u(i) \mid i \leq k \text{ and } u(i) = w(i)\}$.

3. The **pieri** algorithm

Definition 3.1 (k-strip cover, admissible k-chain, the (p,k)-Pieri problem)**.** Let $w \in S_n$, let $1 \leq k < n$, and let $\beta \in \{1, \ldots, n\} \cup \{\infty\}$. A *$k$-strip cover of (w, β)* is a transposition t_{ab} of positions a, b with $1 \leq a \leq k < b \leq n$ such that

(1) $w \lessdot w t_{ab}$ in Bruhat order; that is, $w(a) < w(b)$ and no c with $a < c < b$ satisfies $w(a) < w(c) < w(b)$; and
(2) $w(b) < \beta$.

Its *output* is the pair $(wt_{ab},\, w(b))$. An *admissible k-chain of length p* from u is a sequence $u = w_0, w_1, \ldots, w_p$ carrying bounds $\infty = \beta_0 > \beta_1 > \cdots > \beta_p$ in which (w_t, β_t) is the output of a k-strip cover of (w_{t-1}, β_{t-1}) for every t. The *(p,k)-Pieri problem* for u asks to enumerate every admissible k-chain from u of length at most p. These chains index the permutations occurring in $e_p(x_1, \ldots, x_k)\, \mathfrak{S}_u$, where e_p is the elementary symmetric polynomial.

Method 3.2 (**pieri**(u,p,k))**.** Given $u \in S_n$, $1 \leq k < n$, and $0 \leq p \leq k$, the algorithm returns a list T of pairs (w,t), one for each admissible k-chain from u of length $t \leq p$, recording its endpoint w.

(1) Initialize $T \leftarrow [(u,0)]$ and a frontier $F \leftarrow [(u,\infty)]$.
(2) For $t = 1, \ldots, p$, form a new frontier $F' \leftarrow [\,]$ and, for each $(w,\beta) \in F$:
   (a) compute $A \leftarrow \{\, a \in \{1, \ldots, k\} \mid w(a) < \beta \,\}$;
   (b) for each $b = k+1, \ldots, n$ with $w(b) < \beta$, and each $a \in A$ for which t_{ab} is a k-strip cover of (w,β), append $(wt_{ab},\, w(b))$ to F' and $(wt_{ab},\, t)$ to T.
   Then set $F \leftarrow F'$.
(3) Return T.

Lemma 3.3 (Multiplicity-freeness)**.** *For each t, distinct admissible k-chains from u of length t have distinct endpoints; equivalently, the endpoint map is a bijection from admissible k-chains of length t onto the permutations w with $w \geqslant^k u$ of length $\ell(u) + t$ produced by the Pieri rule for $e_t(x_1, \ldots, x_k)$.*

Proof. The strictly decreasing bounds $\beta_0 > \beta_1 > \cdots$ force the values moved leftward across position k to be strictly decreasing, which is exactly the condition making the e_t-Pieri expansion multiplicity-free; see [2]. Reconstructing the chain from its endpoint by peeling off these moves in decreasing order of value shows the map is injective, hence bijective onto the Pieri support. □

Proposition 3.4 (Complexity of **pieri**)**.** *Let $M := |T|$ be the number of admissible k-chains from u of length at most p that* **pieri**(u,p,k) *returns. The algorithm runs in*

$$O(k\,(n-k)\,n \cdot M)$$

elementary operations, that is $O(n^3)$ per enumerated permutation. Moreover, by Lemma 3.3,

$$M \;\leq\; \#\{\, w \in S_n \mid w \geq u,\ \ell(w) \leq \ell(u) + p \,\}.$$

Proof. Processing one frontier node (w,β) costs $O(k)$ to build A, and then iterates over the $n-k$ choices of b and the at most k elements of A; for each pair the Bruhat-cover test and the transposition each scan $O(n)$ positions. Thus a node costs $O\big(k + (n-k)\,k\,n\big) = O\big(k\,(n-k)\,n\big)$. By Lemma 3.3 the number of nodes ever placed on a frontier equals $\sum_{t=0}^{p-1} N_t$, where N_t is the number of length-t chains; since $\sum_{t=0}^{p-1} N_t \leq \sum_{t=0}^{p} N_t =$

M, the total cost is $O\big(k\,(n-k)\,n\cdot M\big)$. The bound on M holds because every endpoint satisfies $w \geq u$ and $\ell(w) \leq \ell(u) + p$. □

4. The **elemrepr** algorithm

We now describe an algorithm **elemrepr** for expressing a double Schubert polynomial as in Theorem 2.3. Its starting point is the structure of a dominant double Schubert polynomial. If μ is dominant with $\mathfrak{c}(\mu) = (\lambda_1, \ldots, \lambda_m)$, then $\mathfrak{S}_\mu(x; z)$ factors as a product of factorial elementary symmetric polynomials of maximal degree, one per column of the shape,

$$\mathfrak{S}_\mu(x; z) = \prod_{i=1}^{m} E_{\lambda_i,\,\lambda_i}(x; z_i),$$

where z_i is the indicated single indeterminate [8]. For a general v one writes $\mathfrak{S}_v(x; z)$ by applying divided differences in the z-variables to the dominant polynomial $\mathfrak{S}_{\pi^\uparrow(v^{-1})}(x; z)$. By the Leibniz formula, this operation may be carried out one column at a time. Applied to a single column this produces a signed descending expansion.

What **elemrepr** produces is thus a representation of $\mathfrak{S}_v$ as a signed sum of products of elementary symmetric polynomials in nested initial alphabets. A product of this kind is a *standard elementary monomial* (SEM) when its variable counts are strictly decreasing. The representation used here does not require that, and in fact the counts are not determined by v at all: any partition large enough to reach v may serve. The following theorem is the general statement, of which Theorem 2.3 is the minimal case.

Theorem 4.1 (Elementary symmetric representation of a double Schubert polynomial)**.** *Let $v \in S_\infty$ and let λ be a partition. Let μ_λ denote the dominant permutation determined by*

$$\mathfrak{c}(\mu_\lambda^{-1}) = \lambda,$$

and suppose that $v \leq_L \mu_\lambda$ in the left weak order. Then

$$\mathfrak{S}_v(x; z) = \sum_{(v_0, \ldots, v_k)} \prod_{i=1}^{k} (-1)^{\sigma_i(v_{i-1}, v_i)}\, E_{\lambda_i - \ell(v_{i-1}, v_i),\ \lambda_i}\big(x; z_{D_i(v_{i-1}, v_i)}\big),$$

the sum being over all chains $1 = v_0 \overset{1}{\Rightarrow} v_1 \overset{2}{\Rightarrow} \cdots \overset{k}{\Rightarrow} v_k$ with endpoint

$$v_k = v\,\mu_\lambda^{-1}.$$

Proof. Since μ_λ is dominant, so is μ_λ^{-1}, and the dominant double Schubert polynomial factors by columns,

$$\mathfrak{S}_{\mu_\lambda^{-1}}(x; z) = \prod_{i} E_{\lambda_i,\,\lambda_i}(x; z_i),$$

one maximal-degree factorial elementary symmetric polynomial per part of $\lambda = \mathfrak{c}(\mu_\lambda^{-1})$. The hypothesis $v \leq_L \mu_\lambda$ says exactly that $\ell(v\mu_\lambda^{-1}) = \ell(\mu_\lambda) - \ell(v)$, so $\mathfrak{S}_v(x; z)$ is reached from this dominant polynomial by a reduced sequence of $\ell(\mu_\lambda) - \ell(v)$ divided differences in the z-variables, the passage from μ_λ to v being by left multiplication by length-shortening generators. By the Leibniz formula these operators act one column at a time, and on the single column i the effect is the signed descending expansion described above: a step $v_{i-1} \overset{i}{\Rightarrow} v_i$ lowers the degree of the factor E_{λ_i,λ_i} to $E_{\lambda_i - \ell(v_{i-1}, v_i),\,\lambda_i}$, contributes the sign $(-1)^{\sigma_i(v_{i-1}, v_i)}$, and selects the coefficient alphabet $z_{D_i(v_{i-1}, v_i)}$. Composing over the columns and collecting the chains with endpoint $v\mu_\lambda^{-1}$ gives the stated sum. □

The choice of λ is genuinely free, subject only to $v \leq_L \mu_\lambda$. The minimal admissible choice is $\lambda = \theta(v^{-1})$, for which $\mu_\lambda^{-1} = \pi^\uparrow(v^{-1})$ and the endpoint is $v\,\pi^\uparrow(v^{-1})$; the theorem then reduces to Theorem 2.3, and this is the choice **elemrepr** makes, since it minimizes the number of columns and hence the number of layers. Taking λ strictly decreasing recovers the SEM form. All such choices are available, and the next corollary applies to each of them.

Corollary 4.2 (The double upgrade fixes the integer coefficients)**.** *In the notation of Theorem 4.1, setting* $z = 0$ *gives*

$$\mathfrak{S}_v(x) = \sum_{(v_0,\dots,v_k)} \prod_{i=1}^{k} (-1)^{\sigma_i(v_{i-1},v_i)}\, e_{\lambda_i - \ell(v_{i-1},v_i)}(x_1,\dots,x_{\lambda_i}),$$

the sum being over the same chains as in that theorem. The two expansions are indexed by the identical set of chains and attach to each chain the identical integer coefficient $\prod_i (-1)^{\sigma_i(v_{i-1},v_i)}$*; the double polynomial* $\mathfrak{S}_v(x;z)$ *is obtained from the ordinary one* $\mathfrak{S}_v(x)$ *by replacing each factor*

$$e_p(x_1,\dots,x_m) \;\longmapsto\; E_{p,m}\big(x; z_{D_i(v_{i-1},v_i)}\big),$$

that is, by substituting into it the alphabet of coefficient variables indexed by $D_i(v_{i-1}, v_i)$*.*

Proof. Directly from the definition

$$E_{p,m}(x;z) = \sum_{1\le i_1<\cdots<i_p\le m} \prod_{j=1}^{p} (x_{i_j} - z_{i_j-j+1})$$

we have $E_{p,m}(x;0) = e_p(x_1,\dots,x_m)$. Specializing $z = 0$ in Theorem 4.1 therefore replaces each factor $E_{\lambda_i-\ell(v_{i-1},v_i),\,\lambda_i}(x; z_{D_i(v_{i-1},v_i)})$ by $e_{\lambda_i-\ell(v_{i-1},v_i)}(x_1,\dots,x_{\lambda_i})$ and leaves the indexing set and the signs untouched. Since $\mathfrak{S}_v(x;0) = \mathfrak{S}_v(x)$, the displayed expansion follows, and reading the specialization backwards gives the stated substitution rule. □

The consequence for computation is that the z-alphabets are inert combinatorial data: they are selected by the chain, not produced by it. The whole equivariant content of the double case sits inside the substituted factors $E_{p,m}$, which is why **schubmult** and its single-variable specialization traverse identical witnesses with identical signs (Corollary **??**), and why the only cost separating them is that of building one factorial elementary symmetric polynomial (Proposition 5.6).

Remark 4.3 (Stability is independent of the choice of counts). Theorem 4.1 makes the counts a parameter: any partition λ with $v \le_L \mu_\lambda$ furnishes a representation, and Corollary 4.2 applies to each of them, since its proof uses nothing about λ beyond the identity $E_{p,m}(x;0) = e_p(x_1,\dots,x_m)$, valid for every pair $p \le m$. The stability is therefore a property of the shape of the representation rather than of any one choice of counts. Whenever $\mathfrak{S}_v(x;z)$ is written as a signed sum, over some indexing set, of products of factorial elementary symmetric polynomials $E_{p_i,m_i}(x; z_{D_i})$ with integer coefficients, setting $z = 0$ yields the expansion of $\mathfrak{S}_v(x)$ over the same indexing set, with the same integer coefficients, in the products of $e_{p_i}(x_1,\dots,x_{m_i})$; and the double polynomial is recovered from the ordinary one by substituting the designated alphabets z_{D_i} back into the factors. The freedom in the counts is thus genuine rather than a defect: the strictly decreasing SEM counts are admissible, as is the minimal choice $\theta(v^{-1})$ used here, as is every partition between them, and all of them carry the same integer data.

Remark 4.4 (The quantum case). The quantum theory fits the framework of Remark 4.3 with one localized exception, introduced deliberately and only for speed.

Taking the counts strictly decreasing, that is working in the SEM form throughout, everything above carries over: the expansion is a signed sum of products of factorial quantum elementary symmetric polynomials, indexed and signed exactly as in the classical case, and the stability of Corollary 4.2 applies verbatim with those factors in place of the classical ones. No structural obstruction arises. The cost of this uniformity is speed, since the strict counts forgo the savings available from repeated parts.

The optimization recovers those savings. In the classical expansion certain pairs of factorial elementary symmetric factors are consolidated into the double Schubert polynomial of the single permutation they jointly correspond to, replacing two Pieri steps by one. Passing to the quantum setting, in specific cases these consolidated factors may be retained as quantum double Schubert polynomials, which continue to behave correctly under the algorithm. Only these consolidated factors leave the elementary symmetric world: they are not elementary symmetric polynomials unless one specializes $q = 0$, at which point the whole representation reverts to the classical elementary symmetric one. Every unconsolidated factor remains a factorial quantum elementary symmetric polynomial. The almost strict medium shape is what records which pairs have been consolidated.

The Pieri formulas required for the consolidated factors are new and are not proved here; their proofs are substantial enough to require separate treatment, and we forward-cite them to [14] and [13], in preparation. The algorithmic architecture of this paper is unaffected: the layered structure, the one-time v-side precomputation, and the decoupling of the Pieri enumeration from the label count all carry over unchanged, with these formulas substituted for their classical counterparts where a consolidation has been made.

Definition 4.5 (k-strip descent beneath a bound, the down-Pieri problem)**.** Let $w, \mu \in S_n$ and $1 \le k < n$. Say w' lies *beneath the bound* μ if

$$\mathrm{inv}(w'\mu) = \mathrm{inv}(\mu) - \mathrm{inv}(w'),$$

that is, $\ell(w'\mu) = \ell(\mu) - \ell(w')$. A *$k$-strip descent* of w is a Bruhat descent $w \gtrdot wt_{ab}$ realized by a transposition t_{ab}, with $a < b$ and $k \in \{a, b\}$, that moves the value in the pivot position k either leftward (if $b = k$, with sign -1) or rightward (if $a = k$, with sign $+1$), touching only positions still holding their original value. Given a starting label w, the permutation μ, a degree p, and the column k, the *down-Pieri problem* asks for the signed list of those length-t descending k-strip chains from w ($t \le p$) whose endpoint lies beneath μ in weak order.

Method 4.6 (**kdown**(w, μ, p, k))**.** The algorithm returns a list D of triples (w', t, s), one for each length-t descending k-strip chain from w ($t \le p$) whose endpoint w' lies beneath μ in weak order, with $s \in \{+1, -1\}$ its sign.

(1) Initialize $D \leftarrow [\,]$; if w is already beneath μ, append $(w, 0, +1)$. Set a frontier $F \leftarrow [(w, +1)]$.
(2) For $t = 1, \ldots, p$, form $F' \leftarrow [\,]$ and, for each $(w'', s) \in F$:
   (a) (leftward) for each position $a < k$ with $w''(a) = w(a)$ such that $w'' \gtrdot w''t_{a,k}$ is a Bruhat descent, set $w''' \leftarrow w''t_{a,k}$ and append $(w''', -s)$ to F'; if w''' is beneath μ, append $(w''', t, -s)$ to D;
   (b) (rightward) for each position $b > k$ with $w''(b) = w(b)$ such that $w'' \gtrdot w''t_{k,b}$ is a Bruhat descent, set $w''' \leftarrow w''t_{k,b}$ and append $(w''', +s)$ to F'; if w''' is beneath μ, append $(w''', t, +s)$ to D.

   Then set $F \leftarrow F'$.
(3) Return D.

Applied without the bound μ, the signed descending expansion produces many extraneous terms that ultimately cancel. The dominant permutation μ is exactly the bound that discards them: only endpoints beneath μ are retained in D. In the assembled v-side (Method 4.7 below) only these retained labels seed the next column, so the bound prevents the number of live paths from blowing up across columns.

Method 4.7 (**elemrepr**(v), the v-side path dictionaries)**. Input**: Let $\theta := \theta(v^{-1})$ with nonzero parts in columns $1, \ldots, L$, and let $vmu := v\,\pi^{\uparrow}(v^{-1})$.

Output: DAGs $\mathrm{vpathgraphs}_k \subseteq [1, \mathrm{vmu}] \times [1, \mathrm{vmu}]$ for $1 \le k \le L$ such that if $(w', w) \in \mathrm{vpathgraphs}_k$ then $w' \overset{k}{\Longrightarrow} w$, $w' = \mathrm{id}$ if $k = 1$, and $w' \in \{w \mid (w'', w) \in \mathrm{vpathgraphs}_{k-1}\}$ otherwise.

Set $\mathrm{vpathdoms}_L = \{\mathrm{vmu}\}$

For columns from $k = L$ down to $k = 1$, at column k, for T^{k-1} the dominant permutation with $\mathfrak{c}(T^k) = (\theta_1, \ldots, \theta_k)$

For each $w \in \mathrm{vpathdoms}_k$, set

$$\mathrm{vpathfuncs}_k(w) = \mathbf{kdown}(w, T^{k-1}, \theta_k, k)$$

and set

$$\mathrm{vpathdoms}_{k-1} = \mathrm{vpathdoms}_{k-1} \cup \{w' \mid (w', t, s) \in \mathrm{vpathfuncs}_k(w)\}$$

At end: Construct directed graphs $\mathrm{vpathgraphs}_k$ containing (w', w) where $(w', t, s) \in \mathrm{vpathfuncs}_k(w)$.

Note that size of $\mathrm{vpathgraphs}_k$ is at most $\frac{|[1,\mathrm{vmu}]|(|[1,\mathrm{vmu}]|+1)}{2}$ for $1 \le k \le L \le n-1$.

Definition 4.8. For any permutation $u \in S_n$, define

$$\Lambda(u) = \frac{n!}{\prod_{i=1}^{n}(u(i) - \mathfrak{c}_i(u))}$$

Lemma 4.9. *For every $u \in S_n$, we have*

$$\Lambda(u) \le |[e,u]_R|$$

and for every $v \in S_n$ with $u \le_R v$ we have

$$\Lambda(u) \le \Lambda(v)$$

Proof. In the right weak order $w \le_R \sigma$ if and only if $\mathrm{Inv}(w) \subseteq \mathrm{Inv}(\sigma)$, where $\mathrm{Inv}(w) = \{(i,j) \mid i<j,\ w(i) > w(j)\}$. Order $\{1,\dots,N\}$ by the non-inversions of σ, setting $i \prec j$ when $i<j$ and $\sigma(i)<\sigma(j)$; this relation is transitive, and the condition $\mathrm{Inv}(w) \subseteq \mathrm{Inv}(\sigma)$ says exactly that $i \prec j$ implies $w(i)<w(j)$. Hence $|[e,\sigma]_R|$ is the number of linear extensions of the poset P_σ.

We claim P_σ is a rooted forest, that is, the elements above any fixed j form a chain. If not, there are $k<k'$ both above j and mutually incomparable, so $j<k<k'$ with $\sigma(j)<\sigma(k')<\sigma(k)$, an occurrence of the pattern 132; but a permutation is dominant precisely when it is 132-avoiding. By Knuth's hook length formula for forests the number of linear extensions is $N!/\prod_i |{\downarrow} i|$, where ${\downarrow} i = \{j \mid j \preceq i\}$. Since $\prec$ is transitive, ${\downarrow} i = \{j \le i \mid \sigma(j) \le \sigma(i)\}$, of size $1 + \#\{j<i \mid \sigma(j)<\sigma(i)\}$. Finally

$$\sigma(i) - 1 = \#\{j<i \mid \sigma(j)<\sigma(i)\} + \#\{j>i \mid \sigma(j)<\sigma(i)\} = \#\{j<i \mid \sigma(j)<\sigma(i)\} + \lambda_i,$$

so $|{\downarrow} i| = \sigma(i) - \lambda_i \ge 1$, giving the first display. The second follows because $w \mapsto w^{-1}$ is an isomorphism from $[e,\mu]_L$ onto $[e,\mu^{-1}]_R$. Adjoining a fixed point at position $N+1$ contributes the single new factor $N+1$ and replaces $N!$ by $(N+1)!$, leaving the quotient unchanged. □

Proposition 4.10 (Complexity of **kdown/elemrepr**).

Proof. Runtime is

$$\sum_{k=1}^{L} \sum_{w \in \mathrm{vpathdoms}_k} |\mathbf{kdown}(w, T^{k-1}, \theta_k, k)| \le \sum_{k=1}^{L} n^2 \theta_k \Lambda(T^{k-1}) \le n^2 \ell(\pi^\uparrow(v^{-1}))\Lambda(\pi^\uparrow(v^{-1}))$$

□

5. The **schubmult** algorithm

Method 5.1 (The algorithm **schubmult**). Fix n and let $u, v \in S_n$. The algorithm **schubmult**(u,v) computes the expansion of $\mathfrak{S}_u(x;y)\ \mathfrak{S}_v(x;z)$ in the Schubert basis as follows. Set $\theta := \theta(v^{-1})$ and let

$$L := \#\{i \mid \theta_i \ne 0\}$$

be the number of nonzero parts.

The arithmetic takes place in the coefficient ring $R := \mathbb{Z}[y;z]$, in which the structure constants live. The algorithm proceeds in L layers and maintains a hash H that maps each pair (w,λ)—an intermediate u-side permutation w together with a v-side label λ—to a coefficient $c(w,\lambda) \in R$. All coefficients are initialized to 0 except for the single key $(u,\, e)$, which is initialized to $c(u,e) = 1$. Write $\mu := \pi^\uparrow(v^{-1})$ and $vmu := v\mu$.

Each layer transforms H by one column of the shape. Fix layer i and put $k := \theta_i$, and group the keys of H by their u-side permutation. For each distinct u-side permutation w appearing in some key:

(1) Call **pieri**(w, θ_i, θ_i) (Method 3.2) once to enumerate the u-side covers $w \xrightarrow{\theta_i} w'$; each such cover carries a degree $a \in \{0,1,\dots,\theta_i\}$, which is number of inversions it adds. The resulting cover list is reused for every label paired with w and is not recomputed per label.

(2) For each label λ with $c = c(w,\lambda) \ne 0$, read from **elemrepr**(v) (Method 4.7) the v-side transitions out of λ in column i: each is a signed triple $(\lambda',\, b,\, s)$ with endpoint λ', degree b, and sign $s \in \{+1,-1\}$.

(3) For every pair with $w \xrightarrow{\theta_i} w'$ of degree a, triple $(\lambda', b, s))$ with $a+b \le \theta_i$, accumulate into the new hash the single ring element

$$c(w',\lambda') \ \mathrel{+}= \ s \cdot c \cdot E_{\theta_i - a - b,\ \theta_i - a}\big(y_A\,;\, z_B\big),$$

where $E_{p,m}(y;z)$ denotes the factorial elementary symmetric polynomial of degree p in the first m variables,

$$E_{p,m}(y;z) := \sum_{1 \le i_1 < \cdots < i_p \le m} \ \prod_{j=1}^{p} \big(y_{i_j} - z_{\,i_j - j + 1}\big),$$

evaluated on the y-alphabet $y_A := \{\, y_{w'(j)} \mid j \le \theta_i,\ w(j) = w'(j) \,\}$ of the $\theta_i - a$ values the cover leaves fixed in columns $1, \dots, \theta_i$, and the z-alphabet z_B where

$$B = \{i\} \cup \{\lambda(j) \mid j \neq i \text{ and } \lambda(j) = \lambda'(j)\}$$

(4) After all L layers the coefficient $c(w,\, vmu)$ at each u-side endpoint w is the structure constant of $\mathfrak{S}_w(x;y)$ in $\mathfrak{S}_u(x;y)\ \mathfrak{S}_v(x;z)$.

Thus each layer performs, for every matched (cover, triple) pair, the construction of one factorial elementary symmetric polynomial $E_{p,m}(y;z)$, followed by one ring multiplication (by the sign and the running coefficient) and one ring addition into the accumulator $c(w', \lambda')$. The polynomial $E_{p,m}(y;z)$ is rebuilt on each pair by the divide-and-conquer recursion of Lemma 5.2: this recursion is not memoized in the y-alphabet—only the z-index selection is cached—so its construction is a nontrivial cost, recorded as E_n in Definition 5.4, and not a shared lookup. Every coefficient $c(w, \lambda)$ is kept in factored form—a product of the factorial elementary symmetric factors $E_{p,m}(y;z)$ accumulated along its paths—and is never multiplied out into the monomial basis, so it asymptotically stays far smaller than its monomial expansion, whose number of terms is exponential in n.

Lemma 5.2 (Divide-and-conquer splitting of $E_{p,m}$)**.** *Let $y = (y_1, \dots, y_m)$ and let $z = (z_r)_{r\in\mathbb{Z}}$ be an alphabet. For an integer c put*

$$E^{(c)}_{p,m}(y;z) := \sum_{1\le i_1<\cdots<i_p\le m} \prod_{j=1}^{p} \big(y_{i_j} - z_{\,i_j - j + 1 + c}\big),$$

so that $E_{p,m} = E^{(0)}_{p,m}$. Set $t := \lfloor m/2 \rfloor$, $y' := (y_1, \dots, y_t)$, and $y'' := (y_{t+1}, \dots, y_m)$. Then for $0 \le p \le m$ and every integer c,

$$E^{(c)}_{p,m}(y;z) = \sum_{a=0}^{p} E^{(c)}_{a,t}(y';z)\, E^{(c+t-a)}_{p-a,\,m-t}(y'';z),$$

with the conventions $E^{(c)}_{0,\ell} = 1$ and $E^{(c)}_{a,\ell} = 0$ for $a > \ell$, and the base cases $E^{(c)}_{1,\ell}(y;z) = \sum_{i=1}^{\ell}(y_i - z_{i+c})$ and $E^{(c)}_{\ell,\ell}(y;z) = \prod_{i=1}^{\ell}(y_i - z_{1+c})$.

Proof. Classify each index set $1 \le i_1 < \cdots < i_p \le m$ in the defining sum of $E^{(c)}_{p,m}$ by the number a of its members lying in $\{1, \dots, t\}$; the remaining $p-a$ lie in $\{t+1, \dots, m\}$. The first a factors $\prod_{j=1}^{a}\big(y_{i_j} - z_{i_j - j + 1 + c}\big)$ range over all size-a subsets of $\{1, \dots, t\}$ and sum to $E^{(c)}_{a,t}(y';z)$. For the upper part write $i_{a+r} = t + l_r$ with $1 \le l_1 < \cdots < l_{p-a} \le m - t$; the corresponding factor is

$$y_{t+l_r} - z_{\,i_{a+r} - (a+r) + 1 + c} = y''_{l_r} - z_{\,l_r - r + 1 + (c+t-a)},$$

which is the r-th factor of $E^{(c+t-a)}_{p-a,\,m-t}(y'';z)$. So the upper factors sum to $E^{(c+t-a)}_{p-a,\,m-t}(y'';z)$; the terms with $a > t$ contribute 0, consistent with the convention. Summing over a gives the identity. □

Lemma 5.3 (Output support bound)**.** *Let $\pi^{\uparrow}(u,v)$ be the permutation with*

$$\mathfrak{c}(\pi^{\uparrow}(u,v)) = \mathfrak{c}(\pi^{\uparrow}(u^{-1})^{-1}) + \mathfrak{c}(\pi^{\uparrow}(v))$$

a dominant element of S_{2n-1} with $\ell(w_1) = 2\binom{n}{2}$. For any $u, v \in S_n$, every permutation w occurring in the product $\mathfrak{S}_u \cdot \mathfrak{S}_v$, or in any intermediate product formed during its computation, satisfies $w \le_L \pi^{\uparrow}(u,v)$. The number of such permutations is at most $\Lambda(\pi^{\uparrow}(u,v))$.

Proof. □

Definition 5.4 (ℓ_u, ℓ_v, P_u, V, B, Δ, E_n)**.** For $u, v \in S_n$ write $\ell_u := \ell(u)$ and $\ell_v := \ell(v)$, and set

$$P_u := \#\{\, w \mid u \le w \le_L \pi^{\uparrow}(u,v) \text{ and } \ell(w) \le \ell_u + \ell_v \,\},$$

so that every intermediate and output u-side permutation is counted by P_u, and

$$P_u \ \le \Lambda(\pi^{\uparrow}(u,v))$$

Write $\theta := \theta(v^{-1})$, so that $\mu_v^{-1} = \pi^{\uparrow}(v^{-1})$ is the dominant permutation with $\mathfrak{c}(\mu_v^{-1}) = \theta$ (Lemma **??**), and let V be the number of distinct v-side labels produced by **elemrepr**(v) across all columns, so that

$$V \ \le\ \big|[e, v\mu_v^{-1}]\big| \ \le\ \mathrm{md}(v^{-1})!,$$

where an unsubscripted interval denotes Bruhat order. The two *per-layer branchings* are

$$B := \max_i \max_w \big|\mathbf{pieri}(w, \theta_i, \theta_i)\big|, \qquad \Delta := \max_i \max_\lambda \#\{\text{kdown triples } (\lambda', c, \varepsilon) \text{ carried by } \lambda \text{ in column } i\} :$$

B is the maximum number of u-side covers generated by a single **pieri** step (Proposition 3.4), and Δ is the maximum number of v-side kdown triples generated from a single label (Proposition 4.10). Each branching enumerates a family of k-strips indexed by (nonempty) subsets of at most n eligible positions, so

$$B \le 2^n - 1, \qquad \Delta \le 2^n - 1.$$

Finally, let $R(p, m)$ denote the number of coefficient-ring operations (additions and multiplications) performed by the unmemoized divide-and-conquer recursion of Lemma 5.2 in evaluating $E^{(c)}_{p,m}(y; z)$; the count is independent of the offset c. With $t := \lfloor m/2 \rfloor$ it satisfies $R(p, m) = 0$ for $p = 0$ or $p > m$, the base values $R(1, \ell) = R(\ell, \ell) = \ell - 1$, and, for $1 < p < m$,

$$R(p, m) = R(p, t) + R(p, m - t) + 1 + \sum_{a=\max(1,\, p-m+t)}^{\min(p,\, t)} \big(R(a, t) + R(p - a,\, m - t) + 2\big).$$

Define

$$E_n := \max_{0 \le p \le m \le n-1} R(p, m).$$

Remark 5.5. The middle quantity in $P_u \le \big|[e, M]_L\big| \le (2n - 1)!!$ is evaluated exactly by Lemma 4.9, and the outer inequality is attained at $u = v = w_0$. Every v-side label is reached from the initial label $v\mu_v^{-1}$ by a sequence of descending k-strip transpositions, each of which is a Bruhat descent, so all labels lie weakly below $v\mu_v^{-1}$, whence $V \le \big|[e, v\mu_v^{-1}]\big|$. In the resulting chain $V \le \big|[e, v\mu_v^{-1}]\big| \le n!$ the first inequality is the instance-adaptive form and the second the crude bound of Lemma 5.3. The second is usually far from tight, since $\ell(v\mu_v^{-1}) = \ell(\mu_v) - \ell(v)$ collapses as $\ell(v)$ approaches $\ell(\mu_v)$, vanishing outright when v is dominant. The bound $B \le 2^n - 1$ is sharp, attained at $w = w_0$, where $\mathbf{pieri}(w_0, n - 1, n)$ produces exactly $2^n - 1$ covers.

Proposition 5.6. $\log_2 E_n = \Theta(\log^2 n)$*; equivalently* $E_n = 2^{\Theta(\log^2 n)} = n^{\Theta(\log n)}$.

Proof. Write $C(m) := \max_{0 \le p \le m} R(p, m)$, so $E_n = \max_{m \le n-1} C(m)$. Throughout put $t := \lfloor m/2 \rfloor$, so the two child sizes t and $m - t$ lie in $\{\lfloor m/2 \rfloor, \lceil m/2 \rceil\}$, and let N be the number of admissible indices a in the recurrence, so $N \le t + 1 \le m$.

Upper bound. For $1 < p < m$ the recurrence has $2N + 2$ summands of the form $R(\cdot, t)$ or $R(\cdot, m - t)$, each at most $C(\lceil m/2 \rceil)$, together with $2N + 1$ additive constants; hence, using $C(\lceil m/2 \rceil) \ge 1$ for $m \ge 3$,

$$R(p, m) \le (2N + 2)\, C(\lceil m/2 \rceil) + (2N + 1) \le (4m + 3)\, C(\lceil m/2 \rceil).$$

Since $C(2) = 1$, induction on m gives $C(m) \le (4m + 3)^{\lceil \log_2 m \rceil}$: the step uses $4\lceil m/2 \rceil + 3 \le 4m + 3$ and $1 + \lceil \log_2 \lceil m/2 \rceil \rceil = \lceil \log_2 m \rceil$. Therefore

$$\log_2 C(m) \le \lceil \log_2 m \rceil \, \log_2(4m + 3) = O(\log^2 m),$$

and so $\log_2 E_n = O(\log^2 n)$.

Lower bound. Let $I(p, m)$ count the internal nodes—calls with $1 < p < m$—of the unmemoized recursion tree of $R(p, m)$. Each internal node performs at least one ring addition, so $R(p, m) \ge I(p, m)$, and I satisfies, for $1 < p < m$,

$$I(p, m) = 1 + I(p, t) + I(p, m - t) + \sum_{a=\max(1,\, p-m+t)}^{\min(p,\, t)} \big(I(a, t) + I(p - a,\, m - t)\big).$$

Put $J(m) := \sum_{p=0}^{m} I(p, m)$. Dropping all terms of the sum except $I(a, t)$, a fixed value $a^* \in \{1, \dots, t\}$ is an admissible index in the recurrence for $R(p, m)$ precisely when $a^* \le p \le a^* + (m - t)$, which holds for at least $\lfloor m/2 \rfloor$ values of p with $1 < p < m$; summing the resulting contribution $I(a^*, t)$ over those p gives

$$J(m) \;\ge\; \left\lfloor \tfrac{m}{2} \right\rfloor \sum_{a^*=1}^{t} I(a^*, t) \;=\; \left\lfloor \tfrac{m}{2} \right\rfloor J(t).$$

As $J(4) \geq 1$ (since $I(2,4) \geq 1$), unrolling this inequality down to a fixed constant size yields

$$\log_2 J(m) \;\geq\; \sum_{i=0}^{\lfloor \log_2 m \rfloor - 3} \log_2 \left\lfloor \tfrac{m}{2^{i+1}} \right\rfloor \;=\; \Omega(\log^2 m).$$

Finally $J(m) \leq (m+1)\max_p I(p,m) \leq (m+1)\,C(m)$, so $\log_2 C(m) \geq \log_2 J(m) - \log_2(m+1) = \Omega(\log^2 m)$, whence $\log_2 E_n \geq \log_2 C(n-1) = \Omega(\log^2 n)$. □

Theorem 5.7 (Complexity of **schubmult**)**.** *The number of coefficient-ring operations performed by* **schubmult**(u,v) *is*

$$O\big(n^4 \Lambda(\pi^\uparrow(v))^2 N(u, \pi^\uparrow(v)) E_n\big)$$

When v is dominant, this simplifies to

$$O\big(n^4 N(u,v) E_n\big)\,,$$

Proof. We account for the work in two parts. First, the entire v-side is precomputed once by **elemrepr**(v) (Method 4.7) at cost $O(n^2\Lambda(\pi^\uparrow(v)))$ ring operations (Proposition 4.10); thereafter each v-side transition is a table lookup carrying no combinatorial work.

Second, the iterative matched-pair ring work. Write Φ_i for the keys (w, v', s) present at layer i. The inner loop matches one **pieri** cover of w against one kdown (v', s), executed

$$W \;=\; \sum_{i=1}^{L} \sum_{(w,v') \in \Phi_i} \beta_i(w)\,\delta(v')$$

times, where $\beta_i(w) := |\mathbf{pieri}(w, \theta_i, \theta_i)|$ and $\delta_i(v')$ is the number of directed edges with source v' in column i. Bounding $\beta_i(w) = O(n^3)$, $L < n$, and

$$\delta_i(v') = O(\Lambda(\pi^\uparrow(v)))$$

along with

$$\sum_{i=1}^{L} |\Phi_i| = LO(\Lambda(\pi^\uparrow(v)) N(u, \pi^\uparrow(v)))$$

since each w can be visited up to L times gives

$$W = O(n^4 \Lambda(\pi^\uparrow(v))^2 N(u, \pi^\uparrow(v)))$$

Each executed iteration constructs one factorial elementary symmetric polynomial $E_{p,m}(y; z)$ (with $p \leq m \leq \theta_i \leq n-1$) by the unmemoized divide-and-conquer recursion of Lemma 5.2, a fresh evaluation costing up to E_n ring operations (Definition 5.4); the ensuing sign multiplication, ring addition, and $O(n)$ bookkeeping are lower-order. Hence the matched-pair cost is

$$O(W \cdot E_n) = O(n^4 \Lambda(\pi^\uparrow(v))^2 N(u, \pi^\uparrow(v)) E_n)$$

For the dominant case, $\pi^\uparrow(v) = v$ and the v graph collapses to the identity, so the two $\Lambda(\pi^\uparrow(v))$ terms disappear and $N(u, \pi^\uparrow(v)) = N(u, v)$, yielding

$$O(n^4 N(u,v) E_n)$$

as desired.

□

What we would like to show is that the algorithm is output-polynomial. In order to do this, we need to estimate the output size. The following theorem gives necessary and sufficient conditions for a coefficient $c^w_{u,v}(y;z)$ to be nonzero based on the ordinary structure constants c^w_{uv} of the Schubert polynomials $\mathfrak{S}_u(x)\,\mathfrak{S}_v(x)$. This allows us to bound the output size to some extent using the fact that at least one ordinary structure constant is nonzero, with nontrivial bounds arising from this.

Theorem 5.8 (Double support as a union of ordinary supports)**.** *For $u, v \in S_n$ write $c^w_{uv}(y;z)$ for the coefficients in*

$$\mathfrak{S}_u(x;y)\;\mathfrak{S}_v(x;z) = \sum_w c^w_{uv}(y;z)\;\mathfrak{S}_w(x;y),$$

and c_{uv}^w *for the ordinary structure constants of* $\mathfrak{S}_u(x)\ \mathfrak{S}_v(x)$*. Then*

$$c_{uv}^w(y;z) \neq 0 \qquad \Longleftrightarrow \qquad c_{uv''}^w \neq 0 \ \text{ for some } v'' \leq_L v \text{ with } \ell(u)+\ell(v'')=\ell(w),$$

equivalently

$$\operatorname{supp}\big(\mathfrak{S}_u(x;y)\ \mathfrak{S}_v(x;z)\big) \ = \bigcup_{v''\leq_L v} \operatorname{supp}\big(\mathfrak{S}_u(x)\ \mathfrak{S}_{v''}(x)\big).$$

Proof. Fix u and w. We may write

$$c_{u,w_0(n)}^w(x;z) = \sum_v \partial_u^w(\mathfrak{S}_v(x))\,\mathfrak{S}_{vw_0}(-z).$$

If $\ell(u)+\ell(v')=\ell(w)$, then we have the term $c_{u,v'}^w(0;0)\,\mathfrak{S}_{v'w_0(n)}(-z)$ in the sum. We also have that for all v with $vw_0(n) \leq_L v'w_0(n)$, or equivalently $v' \leq_L v$, that

$$\partial_{(z)}^{vw_0(n)}(c_{u,w_0(n)}^w(x;z)) = c_{u,v}^w(y;z).$$

It follows that if $c_{u,v'}^w(0;0)\neq 0$, then $c_{u,v}^w(y;z)\neq 0$ for all $v\geq_L v'$. This proves the direction $\Leftarrow$.

For the converse, suppose for some v that $c_{u,v'}^w(y;z)\neq 0$ for all $v'\geq_L v$. If $\ell(u)+\ell(v)-\ell(w)=0$, then the result is tautological. Otherwise, assume the inductive hypothesis that $c_{u,v}^w(0;z)$ is a nonzero homogeneous polynomial of degree $\ell(u)+\ell(v)-\ell(w)$. Then there exists a Schubert polynomial $\mathfrak{S}_{\widetilde{v}}(-z)$ with degree $\ell(u)+\ell(v)-\ell(w)$ with a nonzero coefficient in $c_{u,v}^w(0;z)$. We may take a right descent s_i of $\widetilde{v}$ and compute

$$\partial_z^i(c_{u,v}^w(y;z)) = -c_{u,s_iv}^w(y;z)\neq 0$$

and conclude that $c_{u,s_iv}^w(0;z)$ is a nonzero homogeneous polynomial of degree $\ell(u)+\ell(v)-\ell(w)-1 = \ell(u)+\ell(s_iv)-\ell(w)$ by properties of divided difference operators. We then conclude that $c_{u,s_iv}^w(y;z)\neq 0$ since the y variables cannot cancel the homogeneous z component, and hence by the previous paragraph that $c_{u,v'}^w(y;z)\neq 0$ for all $v'\geq_L s_iv$, completing the inductive step. This process must eventually terminate at some $v''\leq_L v$ with $\ell(u)+\ell(v'')=\ell(w)$, and we have $c_{u,v''}^w(y;z)=c_{u,v''}^w(0;0)\neq 0$. This proves the direction $\Rightarrow$, and we are done. □

Definition 5.9. Define

$$N^{\uparrow}(u,v) = N(u,\pi^{\uparrow}(v))$$

and

$$N_{\downarrow}(u,v) = N(u,\pi_{\downarrow}(v)).$$

For a Tamari class $\mathcal{C}=[\pi_{\downarrow}(v),\pi^{\uparrow}(v)]_L$ put

$$\widetilde{N}(u,\mathcal{C}) = \frac{1}{|\mathcal{C}|}\sum_{v'\in\mathcal{C}} N(u,v').$$

The classes are the fibres of $v\mapsto\pi^{\uparrow}(v)$, equivalently of $v\mapsto\pi_{\downarrow}(v)$, so each of $N^{\uparrow}(u,-)$, $N_{\downarrow}(u,-)$, $\widetilde{N}(u,-)$ and $|\mathcal{C}|$ takes a single value on a class. Of the quantities appearing below, only $N(u,v)$ itself varies as v ranges over $\mathcal{C}$.

Theorem 5.10. *For each* $u\in S_n$*, the functions*

$$N^{\uparrow}(u,-): S_n\to\mathbb{Z}_{\geq 0}$$

and

$$N_{\downarrow}(u,-): S_n\to\mathbb{Z}_{\geq 0}$$

are weakly increasing with respect to left weak order on S_n*, satisfying*

$$N_{\downarrow}(u,v)\leq N(u,v)\leq N^{\uparrow}(u,v)$$

and constant on Tamari classes, with equality on the left for v 312*-avoiding and on the right for* v *dominant.*

The upper parameter $N^{\uparrow}(u,v)$ is itself an output size. By definition $N^{\uparrow}(u,v) = N(u,\pi^{\uparrow}(v))$ is the number of terms of the product $\mathfrak{S}_u(x;y)\ \mathfrak{S}_{\pi^{\uparrow}(v)}(x;z)$ computed for the dominant permutation $\pi^{\uparrow}(v)$ at the top of the Tamari class of v. The cost bounds below are thus output-polynomial in the honest sense, being polynomial in the size of an actual instance of the problem, differing from the instance (u,v) only in that v is advanced to the top of its class.

Theorem 5.10 fixes the role this parameter plays. Since $N^{\uparrow}(u,-)$ is weakly increasing in left weak order and constant on Tamari classes, it is the maximum of $N(u,v')$ over the class of v, attained at the dominant element $\pi^{\uparrow}(v)$, and it dominates the counts $N(u,v')$ for every v' in every class lying below that of v in the Tamari order. The Tamari class of v is the locus over which the count varies: $N(u,v)$ grows as v ascends toward $\pi^{\uparrow}(v)$, reaching $N^{\uparrow}(u,v)$ at the top, and the runtime is bounded polynomially by that top value throughout.

At the two extremes of the class the v-side Schubert polynomial takes an especially simple closed form, from which the term count is read off directly. The top $\pi^{\uparrow}(v)$ is dominant, equivalently 132-avoiding, and $\mathfrak{S}_{\pi^{\uparrow}(v)}(x;z)$ is a single product of *top-degree* factorial elementary symmetric polynomials, one full factor E_{m_i,m_i} per column of its diagram. The bottom $\pi_{\downarrow}(v)$ is 312-avoiding, so by Theorem 5.10 it realizes the lower count, $N(u,\pi_{\downarrow}(v)) = N_{\downarrow}(u,v)$; when it avoids 1432 as well it is *single-SEM* in the sense of Woodruff [17], meaning $\mathfrak{S}_{\pi_{\downarrow}(v)}(x;z)$ is again a single product of factorial elementary symmetric polynomials, now with at least one factor E_{p_i,m_i} of degree $p_i < m_i$ below top degree, but with the same variable counts m_i as the dominant top of the class. Woodruff proves this for the ordinary polynomial; the double form follows from it together with **elemrepr**. A single SEM is one path through the layered product of **elemrepr**, carrying no signs and admitting no cancellation, so at both ends of the class the count is controlled by the common shape data $\{(p_i,m_i)\}$ and is estimated directly from it.

It is possible to bound $N^{\uparrow}(u,v)$ above nontrivially.

Definition 5.11. Define $\pi^{\uparrow}(u,v)$ to be the dominant permutation with code

$$\mathfrak{c}(\pi^{\uparrow}(u,v)) = \mathfrak{c}(\pi^{\uparrow}(u)) + \mathfrak{c}(\pi^{\uparrow}(v)).$$

Lemma 5.12. *For $u,v \in S_n$ we have*

$$N^{\uparrow}(u,v) \le \Lambda(\pi^{\uparrow}(u,v)).$$

Corollary 5.13. *Fix permutations $u,w \in S_\infty$, with $u \in S_n$. Then the set*

$$\{v \in S_n \mid c^w_{u,v}(y;z) \neq 0\}$$

is a upper order ideal of weak order on S_n.

Corollary 5.14. *Fix permutations $u,w \in S_\infty$, with $u \in S_n$. Then the set*

$$\{v \in S_n \mid c^w_{u,v}(y;z) = 0\}$$

is an ideal of weak order on S_n.

Corollary 5.15 (Output-polynomiality)**.** *For $u,v \in S_n$ we have*

$$T(u,v) = O(n^4\Lambda(\pi^{\uparrow}(v))^2 N^{\uparrow}(u,v)^4 E_n)$$

and

$$T_0(u,v) = O(n^4\Lambda(\pi^{\uparrow}(v))^2 N_0^{\uparrow}(u,v)^4)$$

Moreover, writing

$$W(u,n) := \max_{v\in S_n} T(u,v), \qquad N^*(u,n) := \max_{v\in S_n} N(u,v),$$

we have

$$W(u,n) \le N^*(u,n)^3 \qquad \text{for all } u \in S_n.$$

Proof. If $v = e$ there is nothing to prove, so assume $v \neq e$. By Theorem **??** we have $T(u,v) = O(n^4\Lambda(\pi^{\uparrow}(v))^2 N^{\uparrow}(u,v) E_n)$, and since $N^{\uparrow}(u,v) \ge 1$ this gives the first bound; the second follows the same way from the ordinary estimate of Theorem **??**.

For the last display, the bound $N^*(u,n) \ge n!$ is Proposition **??** at $v = w_0(n)$, where $[e,w_0]_L = S_n$; that proposition is uniform in u, so the single input (u,w_0) witnesses it for whichever u is fixed. The inequality $N^*(u,n) \le W(u,n)$ is that the algorithm must emit each of the $N(u,v)$ coefficients.

For the upper bound, $P_u V \le (2n-1)!!\, n! = (2n)!/2^n = \binom{2n}{n}(n!)^2/2^n \le 2^n (n!)^2$. With $L \le n$, $B\Delta \le 4^n$ and $E_n = 2^{\Theta(\log^2 n)}$ this gives $T(u,v) \le n\, 8^n (n!)^2 E_n$ for every pair, and $n\, 8^n E_n = 2^{3n+\Theta(\log^2 n)} \le n!$ once $\log_2 n > 3 + \log_2 e + o(1)$, by Stirling. Equivalently, at the level of exponents,

$$\log_2\big((n!)^3\big) - \log_2\big(n\, 8^n (n!)^2 E_n\big) = n\log_2 n - (3+\log_2 e)\, n - \Theta(\log^2 n) \longrightarrow +\infty,$$

so the third factor of $n!$ is not consumed and the containment has slack $(n!)^{1-o(1)}$. Combining, $W(u,n) \le (n!)^3 \le N^*(u,n)^3$. □

Remark 5.16 (Scope of the preceding corollary). Since $N^*(u,n)$ is the largest number of coefficients that must be written, it is a lower bound on the worst-case cost, over v, of any algorithm that writes these products; Corollary 5.15 therefore bounds the cost of $\mathbf{schubmult}_n$ by a fixed polynomial in the size of its output, in the worst case over v alone, with u held fixed and arbitrary. Taking the maximum over u as well recovers $W(n) \le N^*(n)^3$.

The pointwise notion of an output-polynomial algorithm, or one running in polynomial total time—running time bounded by a polynomial in the input size plus the output size, on every instance—is due to Johnson, Yannakakis, and Papadimitriou [6]. The last display of Corollary 5.15 is a worst-case relaxation of it: the maximum is taken over v before the comparison, so a large output at one v funds a large running time at every v. It does not assert the pointwise statement $T(u,v) \le N(u,v)^3$, which from the same two bounds would follow only with a correction,

$$T(u,v) \;\le\; N(u,v)^3 \cdot \Big(\frac{n!}{\big|[e,v]_L\big|}\Big)^3,$$

not controlled: since $[e,v]_L \subseteq S_n$, the value $\big|[e,v]_L\big| = n!$ forces $v = w_0$, whereas for v a simple reflection $\big|[e,v]_L\big| = 2$ while $\Lambda(u,v)$ need not fall with it. The pointwise statement is instead supplied by Theorem **??**, which is linear rather than cubic in the output size, but measures that size by $N^{\uparrow}$ and pays the factor $n^4 \Lambda(\pi^{\uparrow}(v))^2$; it is a bound in polynomial total time exactly when the number of columns is bounded.

What the corollary does establish is that the superpolynomial running time of Theorem 5.7 is not an artifact of loose accounting. Since $\log_2 W(u,n) \le 3\log_2 N^*(u,n)$, the exponent is within a factor of three of the information-theoretic floor, for every u separately; and since the input (u,v) occupies $O(n\log n)$ bits while the output may have $n! = 2^{\Theta(n\log n)}$ terms, no algorithm writing these products in the Schubert basis can run in time polynomial in the input length.

Remark 5.17 (Comparison with prior complexity results). The complexity of a single structure constant has been studied more than that of an entire product. For Schur polynomials the coefficients $c^{\nu}_{\lambda\mu}$ are the Littlewood–Richardson coefficients, and computing one of them is $\#P$-complete [10]. Since these coefficients can be exponentially large, a single one has polynomial bit-length yet cannot be evaluated in time polynomial in the binary sizes of the input and output unless FP = $\#P$; the same then holds for the full product $s_\lambda s_\mu = \sum_\nu c^{\nu}_{\lambda\mu}\, s_\nu$. The Littlewood–Richardson rule instead enumerates skew tableaux, of which there are $\sum_\nu c^{\nu}_{\lambda\mu}$, and its running time is polynomial in that number, namely in the coefficients written in unary. Deciding whether a given coefficient is positive is, by contrast, in P, through the saturation theorem [7] and the reduction to hive-polytope nonemptiness [4].

For Schubert polynomials the coefficients c^{w}_{uv} generalize the Littlewood–Richardson coefficients, recovered in the Grassmannian case, so computing one is $\#P$-hard; whether they lie in $\#P$ is open, no combinatorial rule being known. The vanishing problem $\{c^{w}_{uv} = 0\}$ was recently placed in the polynomial hierarchy, in *coAM* assuming the generalized Riemann hypothesis, hence in Σ^p_2 [11].

Since a single coefficient is already $\#P$-hard, neither the bound of Theorem 5.7 nor that of Corollary **??** is polynomial in the binary output size, and the two are compared instead per combinatorial witness. Each executed iteration of **schubmult** processes one matched (cover, triple) pair. Corollary **??** performs $O(1)$ integer operations at each such pair, so in the single case the number of ring operations is proportional to the number of witnesses. Theorem 5.7 performs $E_n = n^{\Theta(\log n)}$ operations at each pair (Proposition 5.6), a quasi-polynomial factor—superpolynomial but subexponential in n—incurred by rebuilding the factorial elementary symmetric weight; the single specialization removes it because that weight collapses to a sign.

Two clarifications are worth making about the Grassmannian bounds of Section 6, since polynomiality there is easy to misread. First, the polynomiality is in the output, not in the input. Corollary 6.5 bounds the cost of the entire product by a degree-four polynomial in G_k, and G_k bounds the number of coefficients

returned. Some such reading is forced: the input λ, μ occupies $O(k \log n)$ bits in binary while the output may carry $\Theta(n^k)$ terms, exponentially many in the input size, so no algorithm for the full product runs in time polynomial in the input, for reasons having nothing to do with $\#P$-hardness. Measured against the output the comparison with the classical rule is favorable: the Littlewood–Richardson rule enumerates $\sum_\nu c^\nu_{\lambda\mu}$ tableaux, which is the coefficient mass in unary and can be exponentially larger than the number of nonzero terms, whereas the bound here is polynomial in the number of terms, with each coefficient carried in factored form. We do not claim asymptotic optimality. Writing the output costs $\Omega(G_k)$ operations in the worst case. In the single case G_k overcounts the support of one product by at most the factor $2k(n-k)+1$ recording the available degrees, that support lying in the single degree $|\lambda|+|\mu|$; in the factorial case no such collapse is available, since by Theorem 5.8 the support is a union of ordinary supports indexed by the left weak order ideal below v and so meets every degree between ℓ_u and $\ell_u + \ell_v$. Either way the information-theoretic floor is linear in G_k up to a polynomial factor, and the gap to the degree-four bound is at most G_k^3. Second, the resulting statement that for fixed k the count is polynomial in n is not itself new, and is in fact weaker than what is known for a single coefficient: the parts of the partitions involved are bounded by $2(n-k)$, so polynomiality in n is polynomiality in the unary size of the input. For a fixed number of rows the hive polytope of [7] has fixed dimension, and Barvinok's lattice-point algorithm [1] evaluates a single Littlewood–Richardson coefficient in time polynomial in the binary size of the partitions; this is stated explicitly in [5]. What Section 6 contributes is therefore not fixed-rank polynomiality but the explicit exponent $4(2n-k)H_2\big(k/(2n-k)\big)$, valid uniformly in k and bounding the cost of the whole expansion by $2^{O(n)}$ rather than $n^{\Theta(n)}$.

6. The Grassmannian case: Schur and factorial Schur multiplication

When both factors are Grassmannian the product specializes to Schur, respectively factorial Schur, multiplication, and the doubled-staircase support of Lemma 5.3 collapses onto a single box. This tightens the counts of Definition 5.4 and, more consequentially, lowers the leading exponent of the running time from $\Theta(n \log n)$ to $4(2n-k)H_2\big(k/(2n-k)\big)$, with H_2 the binary entropy function: a genuine function of the two box dimensions k and $n-k$. The case to keep in mind is the balanced one $k = n/2$, where the relevant count is the central quantity

$$G_{n/2} = \binom{3n/2}{n/2} = (27/4)^{\,n/2+o(n)} = 2.5980\ldots^{\,n(1+o(1))}$$

and the bound G_k^4 reads $(27/4)^{2n+o(n)} = (729/16)^{\,n+o(n)}$, an exponent of $2n\log_2(27/4) = 5.5098\ldots n$. This is essentially the worst case: the maximum of the exponent over all k is $8n\log_2\varphi = 5.5539\ldots n$ for φ the golden ratio, attained at $k \approx 0.553\,n$, less than one percent above the balanced value. The degenerate dimensions are the ones that gain, the exponent vanishing as $k \to 0$ or $k \to n$, and for k held fixed the cost drops to $O(n^{4k})$.

Definition 6.1 (k-Grassmannian permutation)**.** A permutation $w \in S_n$ is *k-Grassmannian* if its unique descent lies at position k, that is $w(1) < \cdots < w(k)$ and $w(k+1) < \cdots < w(n)$ with $w(k) > w(k+1)$. To such a w we associate the partition $\lambda(w)$ with parts

$$\lambda(w)_i = w(k-i+1) - (k-i+1), \qquad 1 \le i \le k,$$

which has at most k parts, each at most $n-k$; that is, $\lambda(w)$ is contained in the $k \times (n-k)$ box. This assignment is a bijection between k-Grassmannian permutations in S_n and partitions in that box.

For a k-Grassmannian u the single Schubert polynomial $\mathfrak{S}_u(x)$ equals the Schur polynomial $s_{\lambda(u)}(x_1, \ldots, x_k)$, and the double Schubert polynomial $\mathfrak{S}_u(x; z)$ equals the factorial Schur polynomial $s_{\lambda(u)}(x; z)$. When u and v are both k-Grassmannian the structure constants of $\mathfrak{S}_u \cdot \mathfrak{S}_v$ are therefore the Littlewood–Richardson coefficients $c^\nu_{\lambda(u)\lambda(v)}$ in the single case [3] and the factorial (Littlewood–Richardson) polynomials in the double case [9, 15].

Lemma 6.2 (Grassmannian output support)**.** *Let $u, v \in S_n$ be k-Grassmannian. Then $\pi^\uparrow(v^{-1})$ is k-Grassmannian, and every permutation occurring in the product $\mathfrak{S}_u \cdot \mathfrak{S}_v$, or in any intermediate product formed during its computation by* **schubmult***, is k-Grassmannian with associated partition contained in the*

$k \times 2(n-k)$ box. Both the u-side permutations and the v-side labels are of this form. The number of such permutations is at most

$$G_k := \binom{2n-k}{k},$$

the number of partitions in the $k \times 2(n-k)$ box.

Proof. That $\pi^{\uparrow}(v^{-1})$ is k-Grassmannian, and that every intermediate u-side permutation and v-side label stays k-Grassmannian, is the structural property of the Grassmannian shape, which we take as given, exactly as the doubled-staircase support was taken as given in Lemma 5.3. It remains to bound the associated partitions. Write $\lambda := \lambda(u)$ and $\mu := \lambda(\pi^{\uparrow}(v^{-1}))$, both contained in the $k \times (n-k)$ box, so $\lambda_1, \mu_1 \le n-k$. Any partition ν arising in the product or a partial product satisfies $\nu \supseteq \lambda$, has at most k parts, and obeys $\nu_1 \le \lambda_1 + \mu_1 \le 2(n-k)$; hence ν is contained in the $k \times 2(n-k)$ box. The number of partitions in a $k \times 2(n-k)$ box is $\binom{k+2(n-k)}{k} = \binom{2n-k}{k} = G_k$. For fixed k this is $\binom{2n-k}{k} = \Theta(n^k)$, a polynomial of degree k in n; uniformly in k it obeys

$$G_k \;\le\; \sum_{j \ge 0} \binom{2n-j}{j} \;=\; F_{2n+1} \;=\; \Theta(\varphi^{2n}),$$

the $(2n{+}1)$-st Fibonacci number, with $\varphi = (1+\sqrt{5})/2$ and $\varphi^2 = (3+\sqrt{5})/2 < 2.619$; hence $G_k = O(\varphi^{2n}) = O(2.619^{\,n})$. Jointly in n and k, Stirling's approximation for binomial coefficients gives the closed form

$$\log_2 G_k = (2n-k)\, H_2\!\left(\frac{k}{2n-k}\right) + O(\log n), \qquad H_2(x) := -x \log_2 x - (1-x)\log_2(1-x),$$

with H_2 the binary entropy function; since $0 \le H_2 \le 1$ this recovers $\log_2 G_k \le 2n-k$. The bound $\binom{m}{k} \le (em/k)^k$ with $m = 2n-k$ gives the sharper reading

$$\log_2 G_k \;\le\; k \log_2 \frac{2n}{k} + k \log_2 e,$$

which for fixed k is $k \log_2 n + O(k)$, matching $G_k = \Theta(n^k)$, and which shrinks to $O(\log n)$ as $k \to 0$; the symmetric bound $\binom{2n-k}{k} \le (k+1)^{2(n-k)}$ gives the same as $k \to n$. Uniformly in k the entropy expression is maximized at $2n \log_2 \varphi$, attained near $k \approx 0.553\, n$, in agreement with the Fibonacci bound above. The running time is governed by the two box dimensions k and $n-k$, never by n alone. Finally, every intermediate shape lies in the same $k \times 2(n-k)$ box, so a single Pieri step produces at most G_k distinct covers; hence the per-step branchings obey $B, \Delta \le G_k$, and the number of layers is at most $n-k$ (the code of v^{-1} has at most $n-k$ nonzero parts). □

Theorem 6.3 (Complexity in the Grassmannian case)**.** *Let $u, v \in S_n$ be k-Grassmannian and set $G_k = \binom{2n-k}{k}$. With $L \le n-k$ layers and per-step branchings $B, \Delta \le G_k$ (Lemma 6.2), the number of coefficient-ring operations performed by* **schubmult**(u, v) *is*

$$O\big(\, n^2\, \Phi_v \;+\; L \cdot G_k \cdot B \cdot n^3 \;+\; L \cdot G_k^2 \cdot B \cdot \Delta \cdot E_n \,\big),$$

with E_n as in Definition 5.4. Writing this bound as $2^{f_{\mathrm{Gr}}(n,k)+O(\log n)}$, the worst case is bounded by the explicit function of n and k

$$f_{\mathrm{Gr}}(n,k) = 4(2n-k)\, H_2\!\left(\frac{k}{2n-k}\right) + \Theta(\log^2 n), \qquad H_2(x) = -x \log_2 x - (1-x)\log_2(1-x),$$

with H_2 the binary entropy function (Lemma 6.2). Equivalently the bound is $O(n\, G_k^4\, E_n)$.

Proof. The three-part accounting of the proof of Theorem 5.7 applies verbatim; only the two counts and the branchings change. By Lemma 6.2 both the u-side permutations and the v-side labels are k-Grassmannian with partition in the $k \times 2(n-k)$ box, so the u-side count P_u and the v-side label count V are each replaced by G_k: the u-side count G_k confines the distinct u-permutations, and the v-side count G_k replaces the general bound $V \le n!$, because the labels are now k-Grassmannian rather than ranging over all of S_n. The hash therefore holds at most G_k^2 keys at once. The live **pieri** enumeration runs once per distinct u-permutation per layer, at most $L \cdot G_k$ times, each $O(n^3 \cdot B)$; the matched-pair work is over $W \le L \cdot G_k^2 \cdot B \cdot \Delta$ pairs, each building one factorial elementary symmetric polynomial at cost E_n; and the v-side precomputation is $O(n^2 \Phi_v)$. Summing gives the stated bound, whose worst-case term is the third.

For the worst case take base-2 logarithms of the third summand. Every shape lies in the $k \times 2(n-k)$ box, so a single Pieri step produces at most G_k covers and $B, \Delta \le G_k$; the number of layers L equals the number of nonzero parts of the code of v^{-1}, at most $n-k$; and $\log_2 E_n = \Theta(\log^2 n)$ (Proposition 5.6). Substituting,

$$\log_2 T \le \log_2(n-k) + 4\log_2 G_k + \Theta(\log^2 n) = 4\log_2 G_k + \Theta(\log^2 n) + O(\log n).$$

By Lemma 6.2, $\log_2 G_k = (2n-k)\, H_2\Big(\frac{k}{2n-k}\Big) + O(\log n)$ with H_2 the binary entropy function, so

$$f_{\mathrm{Gr}}(n,k) = 4(2n-k)\, H_2\left(\frac{k}{2n-k}\right) + \Theta(\log^2 n) \;\le\; 8n\log_2\varphi + \Theta(\log^2 n).$$

The exponent is controlled by the box dimensions k and $n-k$. At $k = n/2$ one has $k/(2n-k) = 1/3$ and $H_2(1/3) = \log_2 3 - \frac{2}{3}$, so $f_{\mathrm{Gr}}(n, n/2) = 6n\, H_2(1/3) + \Theta(\log^2 n) = 2n\log_2(27/4) + \Theta(\log^2 n) = 5.5098\ldots n + \Theta(\log^2 n)$. The exponent collapses to $\Theta(\log^2 n)$ at the extremes $k \to 0$ and $k \to n$, and its maximum over k is $8n\log_2\varphi = 5.5539\ldots n$, attained near $k \approx 0.553\, n$, matching the Fibonacci bound $G_k = O(\varphi^{2n})$ of Lemma 6.2; the balanced value sits within one percent of it. □

Remark 6.4. The general bound of Theorem 5.7 has dominant exponent $\log_2((2n)!) - n = \Theta(n\log n)$, contributed by the $P_u \le (2n-1)!!$ distinct u-side permutations and the $V \le n!$ distinct v-side labels. In the Grassmannian case both counts, and every per-step branching, are confined to the $k \times 2(n-k)$ box and hence bounded by $G_k = \binom{2n-k}{k}$ (Lemma 6.2), so the whole exponent collapses to $4\log_2 G_k = 4(2n-k)H_2\Big(\frac{k}{2n-k}\Big) + O(\log n)$. This is a genuine function of both box dimensions, and its constants are what the collapse consists of. At the balanced dimension $k = n/2$ it is $4\log_2\binom{3n/2}{n/2} = 6n\, H_2(1/3) = 2n\log_2(27/4) = 5.5098\ldots n$, using $H_2(1/3) = \log_2 3 - \frac{2}{3}$, so the cost is $(729/16)^{\,n+o(n)}$; and this is within one percent of its maximum over k, namely $8n\log_2\varphi \approx 5.554\, n$ attained near $k \approx 0.553\, n$; it shrinks to $\Theta(\log n)$ only as $k \to 0$ or $k \to n$, mirroring the shrinking of $\mathrm{Gr}(k,n)$ at the extremes, so the leading exponent $\Theta(n\log n)$ of Theorem 5.7 never appears. The improvement over the general algorithm is a factor $n^{\Theta(n)} = 2^{\Theta(n\log n)}$ in the worst-case bounds. Equivalently the cost is polynomial, of degree at most four, in the number G_k of Grassmannian outputs, and for k held fixed $G_k = \Theta(n^k)$ gives $O(n^{4k})$; Remark 5.17 discusses what such a bound does and does not say.

Corollary 6.5 (Schur multiplication and Littlewood–Richardson coefficients)**.** *Let $u, v \in S_n$ be k-Grassmannian. Computing the structure constants of $\mathfrak{S}_u(x)\ \mathfrak{S}_v(x)$—the Littlewood–Richardson coefficients $c^{\nu}_{\lambda(u)\lambda(v)}$—by the single-variable specialization of* **schubmult** *takes*

$$O\big(\min(n-1, \ell_v)\cdot G_k^2 \cdot B \cdot \Delta\,\big)$$

integer operations, with $L \le n-k$ layers and $B, \Delta \le G_k$. Writing this as $2^{g_{\mathrm{Gr}}(n,k)+O(\log n)}$, the worst case is bounded by the explicit function of n and k

$$g_{\mathrm{Gr}}(n,k) = 4(2n-k)\, H_2\left(\frac{k}{2n-k}\right), \qquad H_2(x) = -x\log_2 x - (1-x)\log_2(1-x),$$

with H_2 the binary entropy function; this is the exponent of Theorem 6.3 with the term $\Theta(\log^2 n)$ deleted.

Proof. As in Corollary **??**, the single-variable weight attached to a matched (cover, triple) pair is the integer sign $s \in \{+1, -1\}$ rather than a factorial elementary symmetric polynomial, so each executed iteration performs $O(1)$ integer operations and E_n is replaced by $O(1)$ in the third summand. The three-part bound of Theorem 6.3 then reads $O(n^2\Phi_v + L \cdot G_k \cdot B \cdot n^3 + L \cdot G_k^2 \cdot B \cdot \Delta)$, and the substitutions $L \le n-k$, $B, \Delta \le G_k$ (Lemma 6.2), $\log_2 G_k = (2n-k)\, H_2\Big(\frac{k}{2n-k}\Big) + O(\log n)$ give $g_{\mathrm{Gr}}(n,k) = 4(2n-k)\, H_2\Big(\frac{k}{2n-k}\Big)$. In particular the operation count is polynomial in the number G_k of Grassmannian outputs, of degree at most four, consistent with Remark 5.17: an individual coefficient $c^{\nu}_{\lambda(u)\lambda(v)}$ may be exponentially large in binary, but the algorithm's arithmetic is counted per output rather than per bit. □

7. Play

$$c^{w}_{u,v}$$

$$c^{w(u^{-1}\pi^{\uparrow}(u))}_{\pi^{\uparrow}(u),v}$$

$$\sum_{v_1,\ldots,v_m} 1$$

where

$$v_0 = v$$

$$v_{i-1} d_i^{-1} \overset{\mathfrak{c}_i^*(\pi^\uparrow(u))+1}{\searrow} v_i$$

$$v_m = \phi_m(wu^{-1}\pi^\uparrow(u))$$

$$d_i = d[\mathfrak{c}_i^*(\pi^\uparrow(u))+1, \mathfrak{c}_i^*(wu^{-1}\pi^\uparrow(u)) - \mathfrak{c}_i^*(\pi^\uparrow(u))]$$

$$d[p,q] = s_p s_{p+1} \cdots s_{p+q-1}$$

8. LR polynomial

Suppose λ, μ, ν are partitions of length at most k. These correspond to permutation $v_k(\lambda), v_k(\mu), v_k(\nu)$. Then for an integer t, the coefficient

$$c_{v_k(t\lambda), v_k(t\mu)}^{v_k(t\nu)}$$

is a polynomial in t. Also,

$$c_{v_k(t\lambda), v_k(t\mu)}^{v_k(t\nu)}(0,z) = \sum_u c_{v_k(t\lambda), uv_k(t\mu)}^{v_k(t\nu)} \mathfrak{S}_u(z)$$

It is sufficient to consider $\mu = (p^k)$ for some p.

$$c_{v_k(t\lambda), v_k((tp)^k)}^{v_k(t\nu)}(0,z) = \sum_u c_{v_k(t\lambda), uv_k((tp)^k)}^{v_k(t\nu)} \mathfrak{S}_u(z)$$

and if we let $u = v_k(t\mu)^{-1}$,

$$c_{v_k(t\lambda), v_k((tp)^k)}^{v_k(t\nu)}(0,z) = \sum_u c_{v_k(t\lambda), v_k(t\mu)^{-1} v_k((tp)^k)}^{v_k(t\nu)} \mathfrak{S}_{v_k(t\mu)^{-1}}(z)$$

$$c_{v_k(t\lambda), v_k((tp)^k)}^{v_k(t\nu)}(0,z) = \sum_u c_{v_k(t\lambda), v_k(t(p^k \setminus \mu^\vee))}^{v_k(t\nu)} \mathfrak{S}_{v_k(t\mu)^{-1}}(z)$$

$$\pi_i = \frac{(1+\beta x_{i+1}) - (1+\beta x_i)s_i}{x_i - x_{i+1}}$$

$$\pi_i = \frac{1 - s_i + \beta(x_{i+1} - x_i s_i)}{x_i - x_{i+1}}$$

$$\pi_i = \partial^i + \frac{\beta(x_{i+1} - (x_i - x_{i+1})s_i + x_{i+1}s_i)}{x_i - x_{i+1}}$$

$$\pi_i = (1+\beta x_{i+1})\partial^i - \beta s_i$$

$$\pi_i = (1+\beta x_i)\partial^i - \beta$$

$$\pi_{i+j}\pi_{i+j-1}\cdots\pi_i$$

$$= \sum_{i \le i_1 < \cdots < i_p \le i+j} (-\beta)^{j-p}(1+\beta x_{i_1})\cdots(1+\beta x_{i_p})\partial^{i_p}\cdots\partial^{i_1}$$

$$\mathfrak{G}_{w_0}(x;y) = \prod_{i+j\le n} (x_i + y_j + \beta x_i y_j)$$

$$(\pi_i + \beta)(fg) = (\pi_i + \beta)(f)s_i(g) + f(\pi_i + \beta)(g)$$

References

[1] Barvinok, A., and Woods, K. Short rational generating functions for lattice point problems. *J. Amer. Math. Soc. 16*, 4 (2003), 957–979.

[2] Bergeron, N., and Sottile, F. Skew Schubert functions and the Pieri formula for flag manifolds. *Trans. Amer. Math. Soc. 354*, 2 (2002), 651–673.

[3] Buch, A. S. A Littlewood–Richardson rule for the K-theory of Grassmannians. *Acta Mathematica 189*, 1 (2002), 37–78.

[4] Bürgisser, P., and Ikenmeyer, C. Deciding positivity of Littlewood–Richardson coefficients. *SIAM Journal on Discrete Mathematics 27*, 4 (2013), 1639–1681.

[5] De Loera, J. A., and McAllister, T. B. On the computation of Clebsch–Gordan coefficients and the dilation effect. *Experiment. Math. 15*, 1 (2006), 7–19.

[6] Johnson, D. S., Yannakakis, M., and Papadimitriou, C. H. On generating all maximal independent sets. *Information Processing Letters 27*, 3 (1988), 119–123.

[7] Knutson, A., and Tao, T. The honeycomb model of $GL_n(\mathbb{C})$ tensor products I: Proof of the saturation conjecture. *Journal of the American Mathematical Society 12*, 4 (1999), 1055–1090.

[8] Macdonald, I. G. *Notes on Schubert Polynomials*. Université Du Québec à Montréal, Dép. de mathématiques et d'informatique, 1991.

[9] Molev, A., and Sagan, B. A Littlewood-Richardson rule for factorial Schur functions. *Trans. Amer. Math. Soc. 351*, 11 (1999), 4429–4443.

[10] Narayanan, H. On the complexity of computing Kostka numbers and Littlewood-Richardson coefficients. *J. Algebraic Combin. 24*, 3 (2006), 347–354.

[11] Pak, I., and Robichaux, C. Vanishing of Schubert coefficients. *arXiv preprint arXiv:2412.02064* (2024). with an appendix joint with David E. Speyer.

[12] Remmel, J. B., and Whitney, R. Multiplying Schur functions. *J. Algorithms 5* (1984), 471–487.

[13] Samuel, M. J. A Pieri formula for multiplying quantum double Schubert polynomials by consolidated pairs of factorial quantum elementary symmetric polynomials. In preparation.

[14] Samuel, M. J. A Pieri formula for quantum double Schubert polynomials. In preparation.

[15] Samuel, M. J. A Molev-Sagan type formula for double Schubert polynomials. *J. Pure Appl. Algebra 228*, 7 (2024), 107636.

[16] Samuel, M. J. `schubmult`: a Python package for multiplying Schubert polynomials. Available from the Python Package Index, `https://pypi.org/project/schubmult/`, with sources at `https://github.com/matthematics/schubmult`, 2026.

[17] Woodruff, D. Single-SEM Schubert Polynomials. *arXiv preprint arXiv:2503.03903* (2025).